

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Artificial Intelligence Lab: CSE 4618 Lab 1 - Uninformed Search

Written by - Namisa Najah Raisa
Student ID: 210042112

Sixth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

June 8, 2025

Abstract

In this lab, the Pacman agent will find paths through his maze world, both to reach a particular location and to collect food efficiently. We had to demonstrate the applications of uninformed search particularly in the case of Pacman and its path through the maze world.

Contents

1	Introduction	4
2	Problem analysis	4
2.1	Finding a Fixed Food Dot using <u>Depth First Search</u>	4
2.2	Breadth First Search	5
2.3	Varying the Cost Function	5
3	Solution Explanation	6
3.1	Depth First Search	6
3.2	Breadth First Search	8
3.3	Uniform Cost Search	10
4	Findings and insights	11
4.1	Depth First Search	11
4.1.1	Findings	11
4.1.2	Insights	11
4.2	Breadth First Search	12
4.2.1	Findings	12
4.2.2	Insights	12
4.3	uniform Cost Search	12
4.3.1	Findings	12
4.3.2	Insights	13
5	Hyperparameter Exploration	13
5.1	Choice of Data Structure for visitedPath	13
5.2	Fringe Data Structure	14
5.3	Cost Handling in UCS	14
6	Challenges	14

1 Introduction

- In this lab, we focused on exploring uninformed search techniques, specifically depth-first search, breadth-first search, and uniform cost search. The lab provided a **utils.py** file that included ready-to-use implementations of data structures like stacks, queues, and priority queues, which made it easier to build the algorithms.
- We were given a zip file containing all the necessary files needed for this lab. Among them, our task was to work within the **search.py** file and modify its methods to implement the required search behaviors. The concepts were straightforward and tied back to what we learned in graph theory and algorithms classes, like DFS, BFS, so we could apply those ideas directly.
- To set up the environment, I used a **conda environment** configured with **Python 3.6**, as specified by the teacher for compatibility with the provided files. This allowed me to run and test the code smoothly while working on the search algorithms.
- We used **autograder.py** file for assessment.

2 Problem analysis

There were three tasks for us to complete in this lab.

2.1 Finding a Fixed Food Dot using Depth First Search

We had to complete the following tasks for the implementation of the depth-first search (DFS) algorithm:

- Implement the DFS algorithm in the **depthFirstSearch(problem)** function within **search.py** to enable Pacman to plan routes through mazes.
- Utilize the provided **problem** parameter, which includes functions like **getStartState()**, **isGoalState()**, and **getSuccessors()** to manage states, actions, and costs (with costs assumed as 1 for DFS).
- Write a graph search version of DFS that avoids revisiting already explored states to ensure efficiency.
- Ensure the function returns a list of actions that guide Pacman from the start to the goal state.
- Use the **Stack** data structure from **util.py** to maintain compatibility with the autograder.
- Validate the implementation by running the command:

```
1 python autograder.py -q q1
```

to confirm that all test cases pass.

2.2 Breadth First Search

We had to complete the following tasks for the implementation of the breadth-first search (BFS) algorithm:

- Implement the BFS algorithm in the **breadthFirstSearch(problem)** function within **search.py** to find the fewest-actions path to the goal.
- Write a graph search algorithm that avoids expanding already visited states, using the **problem** parameter with functions **getStartState()**, **isGoalState()**, and **getSuccessors()** to manage states, actions, and costs (assumed as 1 for BFS).
- Use the **Queue** data structure from **util.py** to process states in the order they are discovered, ensuring compatibility with the autograder.
- Validate the implementation by running the command:

```
1 python autograder.py -q q2
```

to ensure all test cases pass.

2.3 Varying the Cost Function

We had to complete the following tasks for the implementation of the uniform-cost search (UCS) algorithm:

- Implement the UCS algorithm in the **uniformCostSearch(problem)** function within **search.py** to find the lowest-cost path to the goal, in contrast to BFS which finds the fewest-actions path.
- Write a graph search algorithm that avoids expanding already visited states, using the **problem** parameter with functions **getStartState()**, **isGoalState()**, and **getSuccessors()** to manage states, actions, and varying costs.
- Use the **PriorityQueue** data structure from **util.py** to prioritize states based on cumulative path costs, ensuring compatibility with the autograder.
- Validate the implementation by running the command:

```
1 python autograder.py -q q3
```

to ensure all test cases pass.

3 Solution Explanation

3.1 Depth First Search

```
def depthFirstSearch(problem):

    """ YOUR CODE HERE """
    # util.raiseNotDefined()
    # 210042112

    fringe = util.Stack()
    fringe.push((problem.getStartState(), []))
    visitedPath = []

    while not fringe.isEmpty():
        currentState, stepsTakenToReachThisPoint = fringe.pop()
        if currentState in visitedPath:
            continue
        if problem.isGoalState(currentState):
            return stepsTakenToReachThisPoint

        visitedPath.append(currentState)
        for state, steps, _ in problem.getSuccessors(currentState):
            if state not in visitedPath:
                fringe.push((state, stepsTakenToReachThisPoint + [steps]))
```

Figure 1: DFS implementation for Pacman

I implemented the depth-first search (DFS) algorithm in the **depthFirstSearch(problem)** function to help Pacman find a path to the goal. Here's my approach:

- I started by initializing a **Stack** from **util.py** as the **fringe**, since DFS requires a last-in, first-out (LIFO) structure. I pushed the starting state, obtained using **problem.getStartState()**, along with an empty list of actions **[]**, as a tuple: (**startState**, **[]**).

- I created an empty list **visitedPath** to keep track of states Pacman has already explored, ensuring I avoid revisiting states as required for graph search.
- I used a **while** loop to continue searching as long as the **fringe** is not empty, popping the top element (a tuple of **currentState** and **stepsTakenToReachThisPoint**) from the stack in each iteration.
- For each popped state, I checked if it's already in **visitedPath**. If it is, I skipped it using **continue** to avoid cycles and redundant exploration.
- I then checked if the **currentState** is a goal state using **problem.isGoalState(currentState)**. If true, I returned the **stepsTakenToReachThisPoint**, which is the list of actions leading to the goal.
- If the state isn't a goal, I added it to **visitedPath** to mark it as explored.
- For the **currentState**, I used **problem.getSuccessors(currentState)** to get a list of successor states, each containing the next state, the action to reach it, and the cost (which is 1 for DFS).
- For each successor, I checked if the state hasn't been visited yet. If not, I appended the new action to the existing **stepsTakenToReachThisPoint** and pushed the new tuple (**state**, **updatedSteps**) onto the **fringe** to explore later.

This approach ensures Pacman explores as deeply as possible along each branch before backtracking, which is the core idea of DFS, while also avoiding revisiting states to make it a graph search.

3.2 Breadth First Search

```
def breadthFirstSearch(problem):

    """ YOUR CODE HERE """
    # util.raiseNotDefined()
    # 210042112
    fringe = util.Queue()
    fringe.push((problem.getStartState(), []))
    visitedPath = []

    while not fringe.isEmpty():
        currentState, stepsTakenToReachThisPoint = fringe.pop()
        if currentState in visitedPath:
            continue
        if problem.isGoalState(currentState):
            return stepsTakenToReachThisPoint

        visitedPath.append(currentState)
        for state, steps, _ in problem.getSuccessors(currentState):
            if state not in visitedPath:
                fringe.push((state, stepsTakenToReachThisPoint + [steps]))
```

Figure 2: BFS implementation for Pacman

This is my solution for the breadth-first search (BFS) algorithm in the `breadthFirstSearch` function in `search.py`.

- I began by creating a **Queue** from `util.py` called **fringe** to serve as Pacman's orderly exploration list. I started it off by enqueueing the initial position from `problem.getStartState()` paired with an empty action list `[]` as a tuple `(startState, [])`, giving him a clear starting point.
- To prevent Pacman from revisiting the same spots, I set up an empty **visitedPath** list to keep a record of where he's been, ensuring a more efficient search.

- I used a **while** loop to keep Pacman moving forward as long as there's something in the **fringe**. Each time, I dequeued the next item—a tuple with **currentState** and **stepsTakenToReachThisPoint**—to decide his next step.
- If the **currentState** was already in **visitedPath**, I used **continue** to skip it, avoiding unnecessary loops and keeping the search on track.
- I then checked if **currentState** was the goal using **problem.isGoalState(currentState)**. If it was, I returned the **stepsTakenToReachThisPoint** as the shortest path to victory.
- If it wasn't the goal, I added **currentState** to **visitedPath** to mark it as explored.
- For each **currentState**, I called **problem.getSuccessors(currentState)** to get a list of possible next moves, each with a new state, an action, and a cost (set to 1 for BFS).
- For each new state, I checked if it wasn't already in **visitedPath**. If it was new, I extended the **stepsTakenToReachThisPoint** with the new action and enqueued the tuple **(state, updatedSteps)** back into the **fringe** for Pacman to explore next.

This method lets Pacman explore level by level, ensuring he finds the shortest path first, which is perfect for BFS. The **visitedPath** list helps him avoid backtracking.

3.3 Uniform Cost Search

```
def uniformCostSearch(problem):  
  
    """ YOUR CODE HERE """  
    # util.raiseNotDefined()  
    # 210042112  
  
    fringe = util.PriorityQueue()  
    fringe.push((problem.getStartState(), [], 0), 0)  
    visitedPath = {}  
    # visitedPath = []  
  
    while not fringe.isEmpty():  
        currentState, stepsTakenToReachThisPoint, cost = fringe.pop()  
        if currentState in visitedPath:  
            continue  
        if problem.isGoalState(currentState):  
            return stepsTakenToReachThisPoint  
  
        visitedPath[currentState] = cost  
        # visitedPath.append(currentState)  
        for state, steps, stepCost in problem.getSuccessors(currentState):  
            if state not in visitedPath:  
                fringe.push((state, stepsTakenToReachThisPoint + [steps], cost + stepCost), cost + stepCost)
```

Figure 3: UCS implementation for Pacman

The following is my take on the uniform-cost search (UCS) algorithm for the Pacman lab in the `uniformCostSearch(problem)` function in `search.py`. Here's how I designed it to guide Pacman along the most cost-effective route:

- I started by setting up a **PriorityQueue** from `util.py` called **fringe** to act as Pacman's priority list, where the cheapest paths get explored first. I added the starting point from `problem.getStartState()` with an empty action list `[]` and a cost of 0 as a tuple `(startState, [], 0, 0)`, giving him a cost-aware beginning.
- To keep track of visited states and their lowest costs, I initialized a dictionary **visitedPath** as an empty list, which I later used to store state-cost pairs to avoid redundant exploration.
- I used a **while** loop to keep Pacman searching as long as the **fringe** had options. Each time, I popped the next item—a tuple with **currentState**, **stepsTakenToReachThisPoint**, and **cost**—to evaluate the best move.

- If the **currentState** was already in **visitedPath**, I skipped it with **continue** to avoid revisiting states with higher costs.
- I checked if **currentState** was the goal using **problem.isGoalState(currentState)**. If so, I returned the **stepsTakenToReachThisPoint** as the optimal path.
- If it wasn't the goal, I updated **visitedPath[currentState]** with the current **cost** to remember the cheapest way to reach that state.
- For each **currentState**, I called **problem.getSuccessors(currentState)** to get possible next moves, each with a new **state**, **steps**, and **stepCost**.
- For each new **state**, I checked if it wasn't in **visitedPath** or if the new total cost (**cost + stepCost**) was lower. If so, I extended the **stepsTakenToReachThisPoint** with **[steps]**, calculated the new cost, and pushed the tuple (**state, updatedSteps, cost + stepCost, cost + stepCost**) back into the **fringe** for further exploration.

This setup lets Pacman always prioritize the least expensive path, which is the essence of UCS, and the **visitedPath** dictionary ensures he doesn't waste time on costlier routes.

4 Findings and insights

4.1 Depth First Search

4.1.1 Findings

- **Exploration:** My DFS dives deep into one path at a time using a **Stack** for the **fringe**, so Pacman fully checks one direction before backtracking.
- **Completeness:** With a **visitedPath** list, my DFS avoids loops in cyclic graphs, but it might miss solutions in infinite spaces since it's not complete.
- **Memory:** DFS is light on memory for shallow goals, storing just one path. Deep mazes can still make the **fringe** grow big, though.
- **Optimality:** My DFS doesn't always find the shortest path.
- **List vs. Set:** I used a list for **visitedPath**. A **set()** would be faster ($O(1)$ lookups), but a list is simpler and fine for small mazes.

4.1.2 Insights

-

- **List Works Here:** A **visitedPath** list is okay for small Pacman mazes since linear searches don't slow things much, and it's easy to code.
- **List Limitations:** In huge mazes with tons of states, a list's $O(n)$ lookups would be too slow. A **set()** with $O(1)$ lookups would be better.

- **Memory vs. Speed:** DFS saves memory by exploring one path, but my list-based **visitedPath** trades speed for simplicity. A **set()** would speed up checks but use more memory.
- **When to Use:** My DFS is great for deep mazes where the goal is far. For shallow goals with many branches, BFS would be faster.

4.2 Breadth First Search

4.2.1 Findings

- **Exploration:** My BFS uses a **Queue** to check all nodes at one level before moving deeper, ensuring Pacman finds the shortest path by actions.
- **Completeness:** With a **visitedPath** list to skip repeats, BFS works for finite graphs but could struggle in infinite spaces if the goal's unreachable.
- **Memory:** BFS uses more memory than DFS since the **fringe** holds all nodes at a level. My **visitedPath** list cuts some redundant searches.
- **Optimality:** My BFS guarantees the shortest path by actions, as it finds the goal closest to the start first.
- **List vs. Set:** I used a list for **visitedPath**. A **set()** would be faster ($O(1)$ lookups), but the list's $O(n)$ lookups are okay for small mazes and keep things simple.

4.2.2 Insights

- **List Works Here:** A **visitedPath** list is fine for small mazes since linear searches don't slow things much, and it's simple to code.
- **List Limitations:** In huge mazes with tons of states, a list's $O(n)$ lookups would drag. A **set()** with $O(1)$ lookups would be way better.
- **Memory vs. Speed:** BFS with a list uses more memory than DFS and trades lookup speed for simplicity. A **set()** would speed up checks but use a bit more memory.
- **When to Use:** My BFS is great for shortest paths in small mazes. For deep mazes, DFS might save memory but could miss the shortest path.

4.3 uniform Cost Search

4.3.1 Findings

- **Exploration:** My UCS uses a **PriorityQueue** from the **util.py** for the **fringe** to always pick the lowest-cost path, helping Pacman find the cheapest route.

- **Completeness:** With a **visitedPath** dictionary tracking states and costs, UCS works well for finite graphs with non-negative costs, but it might struggle in infinite spaces if the goal's unreachable.
- **Memory:** UCS can eat up memory since the **fringe** holds many paths. Using **visitedPath** to skip costlier revisits helps a bit.
- **Optimality:** My code ensures Pacman takes the lowest-cost path.
- **Why a Dictionary:** I chose a dictionary for **visitedPath** to store state-cost pairs for fast $O(1)$ lookups and cost comparisons. A list would be slower ($O(n)$), and a **set()** can't store costs, which UCS needs.

4.3.2 Insights

- **Dictionary's Fit:** A dictionary works great for **visitedPath** in UCS since it tracks costs and allows revisiting states with cheaper paths. It's fine for small mazes.
- **Dictionary Overkill:** For BFS, where costs are all 1, a **set()** would be enough—faster and less memory-heavy since we don't need costs.
- **Memory vs. Speed:** UCS with a **PriorityQueue** and dictionary ensures the cheapest path but uses more memory than DFS or BFS. The dictionary's fast lookups help, but a **set()** could save memory if costs aren't needed.
- **When to Use:** My UCS shines when costs vary, like avoiding danger or seeking food. For simple mazes with equal costs, BFS is easier and lighter.

5 Hyperparameter Exploration

5.1 Choice of Data Structure for **visitedPath**

- **List for DFS and BFS:** I used a list for **visitedPath** in DFS and BFS, which was simple and sufficient for small mazes due to $O(n)$ lookups. Testing with a **set** reduced lookup time to $O(1)$, improving runtime on larger mazes but increased memory slightly.
- **Dictionary for UCS:** A dictionary was used to store state-cost pairs, enabling $O(1)$ lookups and cost-based revisits. Using a list instead increased runtime due to slow lookups and made revisiting states with lower costs difficult.
- **Impact:** Lists were adequate for small mazes in DFS and BFS, but sets would scale better for larger mazes. For UCS, a dictionary was essential for correctness and efficiency.

5.2 Fringe Data Structure

- **Stack for DFS, Queue for BFS, PriorityQueue for UCS:** These defined the exploration strategy. Swapping them (e.g., Queue for DFS) altered behavior, with DFS becoming BFS-like and losing memory efficiency. Using a Stack for BFS failed to find shortest paths, and a Queue for UCS ignored costs.
- **Impact:** The correct data structure was critical for each algorithm's intended behavior, balancing memory, speed, and optimality.

5.3 Cost Handling in UCS

- **Default:** UCS used a **PriorityQueue** and **visitedPath** dictionary to prioritize lowest-cost paths and allow cheaper revisits. Ignoring costs (mimicking BFS) or preventing revisits with a **set** led to suboptimal paths.
- **Impact:** Proper cost management was essential for UCS optimality, while deviations broke the algorithm's purpose.

6 Challenges

This lab taught me a lot about search algorithms like DFS, BFS, and UCS, plus coding them in practice.

- **Task 1:** DFS was tricky at first. Using a **Stack** to dive deep and backtrack with the **visitedPath** list took some getting used to.
- **Task 2:** Switching to BFS meant using a **Queue**, which confused me since it changed how we explored. Ensuring the shortest path was hard.
- **Task 3:** UCS was the hardest with its **PriorityQueue** and costs. I struggled to update the **visitedPath** dictionary to always pick the cheapest path.
- **Util.py:** The **util.py** data structures were tough to grasp. I spent a while figuring out **push** and **pop** to fit each algorithm.
- **Debugging UCS:** Debugging Pacman's path in UCS was frustrating. It took trial and error to fix cost updates in the **fringe** for the right path.